

Wormhole

Detailed concept reference for the completed-picture version of the app

What this document is

This is a concept architecture document. It explains how Wormhole should be understood as a complete product: what the three major systems are, what each system is responsible for, how they connect, what the user experience should feel like, and where the boundaries should stay so the app does not turn into a bloated mess. It is intentionally more detailed than a pitch, but less implementation-heavy than a build spec.

Core idea. Wormhole is a personal universe explorer and story-management app. Each “wormhole” is one separate world, story universe, or creative space. Opening a wormhole should feel like entering a coherent fictional world rather than opening a pile of folders, notes, or disconnected assets.

The app is organized around **three primary systems**. Those three systems are not equal side features. Together, they form the whole product:

- **Atlas Canvas** — the world map and spatial system
- **Character Codex** — the character profile and character-detail system
- **Lore Archive** — the longform lore, writing, note, and document system

The main design principle behind Wormhole is simple: **high expression with enough structure to prevent chaos**. The app should let the user pour a lot of creative detail into a world, but that detail must remain navigable, linkable, and maintainable over time.

1. Wormhole at a glance

At the highest level, Wormhole is not just a note app, not just a map app, and not just a character database. It is a **world-centered creative system** that combines spatial world representation, structured character records, and longform lore documents into one local, connected environment.

A useful mental model is this:

Finished-picture mental model

A wormhole is a world. Inside that world, Atlas Canvas shows where things are, Character Codex shows who matters, and Lore Archive holds the deeper written material. The app becomes valuable when the user can move naturally between those three layers without getting lost.

System	Main role	Primary object type	What the user mainly does there
Atlas Canvas	Represents the world spatially	Map entities such as regions, settlements, landmarks, structures	Place, inspect, and connect meaningful places inside the world

System	Main role	Primary object type	What the user mainly does there
Character Codex	Represents people and identity inside the world	Character records with structured core fields and optional modules	Create, understand, compare, and link characters
Lore Archive	Stores deep written content	Lore documents, notes, chapters, diaries, summaries	Write, organize, and cross-link longform world/story material

What Wormhole is **not** supposed to become:

- not a full game engine
- not a social platform or collaborative workspace
- not a general-purpose PKM tool with no world-specific structure
- not a drawing suite, sprite editor, or animation package in v1/v1.5 thinking
- not a place where every object in the UI can become a deeply modeled system just because it seems cool

2. The three-system architecture

Wormhole should be read through a three-system structure first. That framing matters because the concept becomes confusing when everything is described as one giant creative blob. The app does contain many moving parts, but they should resolve into three clear containers of responsibility.

2.1 Atlas Canvas

Atlas Canvas is the **spatial layer** of the app. It is the place where the user sees the world as a world: regions, countries, settlements, landmarks, infrastructure, and other meaningful locations. It should feel like a world-facing canvas, not a spreadsheet.

- **Main job:** make the world visible and explorable
- **Primary value:** lets a place be more than text; it becomes a navigable entry point into the world
- **Core rule:** not every visual object needs deep data; decorative objects and semantic world objects must stay separate

2.2 Character Codex

Character Codex is the **identity layer** of the app. It holds the people who matter inside the world, their roles, traits, affiliations, summaries, relationships, and links into deeper material. It should feel like a structured creative codex, not like a rigid HR form and not like a fully freeform scrapbook.

- **Main job:** let the user understand who a character is, why they matter, and how they connect to the world
- **Primary value:** combines stable structure with room for expressive flavor
- **Core rule:** use a hybrid model — fixed core profile, optional modules, and limited freeform expression

2.3 Lore Archive

Lore Archive is the **document layer** of the app. It stores deeper written content: lore notes, summaries, chapter material, event writeups, journals, diaries, world history, and similar longform content. It should feel closer to an Obsidian-like note archive than to a database form, but it must still connect cleanly to the structured parts of the app.

- **Main job:** preserve the longform writing and reference material of the world
- **Primary value:** gives depth without overloading the character or map systems with massive text blobs
- **Core rule:** the archive stores writing; it does not become the hidden database for the whole app

Boundary rule

Atlas Canvas handles places. Character Codex handles people. Lore Archive handles deep writing. They link to each other, but they should not absorb each other's jobs. That separation is what keeps Wormhole understandable.

2.4 Support layers that exist across all three systems

Even though Wormhole is built around three primary systems, some support layers sit across the whole app:

- **World container:** each wormhole is the top-level world/project container
- **Links and relationships:** records can point to related characters, places, and documents

- **Search and navigation:** the user should be able to find things without remembering exact locations
- **Visual identity:** world cover image, accents, optional decoration, and presentation cues

Layer	What it governs	Why it matters
World container	Keeps each universe separate	Prevents mixing unrelated worlds and gives the app a clean top-level structure
Links/relations	Cross-system connections	Lets a character connect to locations, events, and lore documents without duplicate writing
Search/navigation	Fast retrieval	Makes the app usable when a world becomes large
Visual identity	Presentation and flavor	Supports expression without replacing structure

3. Atlas Canvas — the world map and spatial system

Atlas Canvas is the first major system because it is the part that gives Wormhole a strong identity. It is the reason the app can feel like entering a world rather than browsing a folder tree. The important point is that Atlas Canvas is not merely a static image viewer. It is a **visual world layer with data-bearing entities**.

3.1 What Atlas Canvas is

Atlas Canvas is a zoomable, pannable world space where the user can see and interact with meaningful locations inside a world. It is responsible for representing geography and place-based structure at a level that is useful for worldbuilding and story navigation.

- The canvas may use a background map image, a blank world canvas, or a simple placed-layout model depending on the eventual version.
- Large world entities such as continents, countries, regions, and settlements can function as structured map entities.
- Meaningful placed locations such as landmarks, districts, buildings, or infrastructure can appear within those larger entities.
- Selecting a place should open associated information rather than just highlighting a shape.

3.2 What Atlas Canvas is not

- not a full GIS tool
- not a simulation of logistics, adjacency, travel time, or politics unless explicitly added later
- not a mandatory freehand art engine at first
- not a system where every tree, rock, or decorative icon must become a fully structured record

3.3 Two kinds of objects inside the canvas

One of the most important design decisions for Atlas Canvas is the split between **decorative objects** and **semantic world objects**. Without this distinction, the map system quickly becomes bloated and inconsistent.

Object category	Examples	How the app should treat it
Decorative visual objects	trees, rocks, flourishes, non-meaningful shapes, aesthetic markers	Mostly visual. May store position and style, but should not force deep world data unless deliberately promoted into a semantic object.
Semantic world objects	region, country, settlement, district, landmark, historical site, building, infrastructure	Structured entities with identity, description, links, and relationships to characters and lore docs.

3.4 Types of semantic map entities

The map layer should not force every world to use the same exact hierarchy, but it still needs a sensible base vocabulary. A practical set of semantic map entities would be:

- **Region-level entities:** continent, country, kingdom, province, territory, district
- **Settlement-level entities:** city, town, village, encampment, colony
- **Landmark-level entities:** mountain, forest, ruin, academy, temple, port, bridge, battlefield, historic site

- **Structure-level entities:** major building, infrastructure, significant residence, fortress, station, palace

The point of this hierarchy is not realism for its own sake. It is to give the user a clean way to represent places at the level that matters narratively.

3.5 Data attached to a semantic place

A place inside Atlas Canvas should be more than a title and a description. It should act as a place record that can connect outward to the rest of the world.

- **Identity:** title, type, optional parent place, optional visual symbol
- **Description:** summary or short lore description
- **Spatial data:** position, size, optional region boundary reference, z-order or layer if relevant
- **Linked characters:** through typed relationships such as resident, ruler, origin, active in, tied to event in
- **Linked lore:** linked notes, linked events, linked historical records, linked references

3.6 Navigation behavior

The canvas should support exploration rather than just editing. The user should be able to pan, zoom, select a place, and then open a side panel or detail pane that reveals what that place is, who is associated with it, and what writing is connected to it.

- Map selection should reveal structured details.
- The place panel should expose related characters and related lore docs.
- The user should be able to jump from a place into Character Codex or Lore Archive without losing orientation.

Key Atlas Canvas rule

The map is an entry point into the world, not a dumping ground for every possible system. If a detail does not improve place-based understanding, it probably does not belong in Atlas Canvas.

4. Character Codex — the character profile and detail system

Character Codex is the second major system and likely the most emotionally expressive one. It is where the user understands who a character is, how that character reads, what role they play, and where they connect to the world. The trap here is going too rigid or going too freeform. The product should avoid both extremes.

4.1 Core purpose

- Preserve a stable character record that can be searched, compared, and maintained over time
- Allow enough flavor and expressive text that the character still feels alive
- Support links outward to places, lore documents, and related people

4.2 The recommended hybrid structure

The strongest model for Character Codex is a three-layer character page. This is the compromise between rigid schema and total block chaos.

Layer	Purpose	Typical content
Layer A — fixed core profile	Stable, app-understood data	name, image, role, affiliations, tags/traits, quick factual summary
Layer B — optional structured modules	World-specific but still organized data	relationships, linked locations, important events, arc notes, abilities, status fields, optional affiliations
Layer C — freeform expressive section	Flavor and personal voice	in-character intro, custom notes, atmosphere text, short freeform blocks, personal observations

4.3 Why the hybrid model matters

Purely custom character pages sound flexible, but they create long-term problems: searching becomes weaker, filtering becomes weaker, consistency disappears, and creating new pages becomes more exhausting than it first appears. On the other hand, a giant fixed form with every possible field creates clutter and empty sections. The hybrid model solves both problems by keeping the core stable while allowing optional modules and controlled freeform expression.

4.4 Fixed core profile

- **Name** — primary character identity
- **Primary image** — main visual reference
- **Role** — concise statement of narrative or world role
- **Affiliations** — membership, alignment, or key association where relevant
- **Traits/tags** — short searchable descriptors
- **Objective summary** — factual paragraph about who they are

4.5 Optional structured modules

Modules should be enabled when they matter to the world or the character. This avoids empty clutter while preserving structure.

- **Relationships module** — related characters and typed relationship notes
- **Places module** — origin, residence, active areas, or important linked places
- **Events module** — important events or incidents connected to the character
- **Arc module** — key change points or character-development notes
- **Reference links module** — direct links into relevant lore documents

4.6 Freeform expression

The expressive part should exist, but it should be given a clear role rather than replacing the whole profile. A strong example is the **in-character introduction**: a paragraph written as if the character were speaking or presenting themselves. This gives flavor, tone, and immersion without forcing the rest of the page to abandon structure.

- Objective summary and in-character intro should be distinct.
- Inline links inside prose are allowed as a flavor layer, but quick links should still exist outside the prose.
- Freeform text should enrich the profile, not become the only way important information is stored.

4.7 Decorative elements

Decorative ribbons, accents, and optional visual flourishes may exist, but they should be treated honestly as decoration unless the app explicitly models them as data. In the current refined concept, ribbons are **purely decorative**. They can carry personal meaning for the user, but the app should not rely on them as formal faction logic.

4.8 What Character Codex should not become

- not a full custom page-layout builder for every profile
- not a mini desktop publishing tool
- not a substitute for all longform writing
- not a place where every tiny visual preference becomes a core feature early

5. Lore Archive — the longform lore and writing system

Lore Archive is where the world breathes in writing. If Atlas Canvas provides place and Character Codex provides identity, Lore Archive provides depth. It stores the material that would be too large, too narrative, or too document-like to fit comfortably inside the other two systems.

5.1 Core purpose

- Store longform worldbuilding and narrative material
- Let the user maintain notes, summaries, scenes, event files, history pages, diary pages, and chapter-like documents
- Support internal linking and cross-reference without forcing every piece of writing into a rigid schema

5.2 What lives inside Lore Archive

- **Lore notes** — setting details, customs, systems, history, mythology, culture
- **Event pages** — incident descriptions, battles, ceremonies, turning points, discoveries
- **Character-linked documents** — diaries, private notes, extended backstory, records tied to a specific person
- **Story material** — chapter notes, summaries, scene concepts, snippets, or longer prose pieces

5.3 Document structure

The intended feel is close to a lightweight Obsidian-style archive: folders and documents, with linkability between documents. The archive should support internal organization, but it should not become so loose that nothing has a home.

- Documents can be grouped by folders or categories.
- Internal links between documents should be supported.
- Documents can also be linked outward from characters and places.
- The archive can contain both concise reference notes and more narrative pieces.

5.4 Boundary with the rest of the app

The key boundary is this: **Lore Archive stores writing, but it is not the source of truth for all app data.** The structured entities of the app should remain structured. Character records stay character records. Place records stay place records. Lore documents can describe, expand, or contextualize those entities, but should not secretly replace them.

5.5 Why that boundary matters

- Prevents the app from becoming a confusing mix of documents and hidden internal state
- Keeps longform writing free and expressive without breaking structured search and retrieval
- Makes future evolution cleaner because the archive remains content-facing while the app data remains system-facing

5.6 Reading and linking behavior

A lore document should not feel isolated. When opened, the user should be able to see related characters, related places, and related documents. Likewise, a character page or place page should be able to surface linked archive items as quick links or related records.

- Inline links inside lore text are allowed and useful.
- A related-records panel is valuable for discoverability.
- Pop-up previews are optional UX sugar; they are not the conceptual core.

5.7 What Lore Archive should not become

- not a clone of every Obsidian feature under the sun
- not a hidden storage folder for internal database JSON
- not the only place where important truth exists

Lore Archive rule

Use the archive for depth, context, and writing. Use the structured systems for identity and structure. The app stays understandable only if those two jobs stay separate.

6. How the three systems connect

The value of Wormhole is not that it has three systems. The value is that those three systems **resolve into one usable world**. A user should be able to move across space, people, and writing without feeling like they switched applications.

6.1 Connection pattern

From	To	Example connection	Why it matters
Atlas Canvas	Character Codex	A capital city links to its ruler and important residents	Makes the map feel inhabited rather than empty
Atlas Canvas	Lore Archive	A historic battlefield links to an event document	Lets places open into deeper context
Character Codex	Lore Archive	A diary link or backstory note opens a related lore document	Supports depth without overloading the profile itself
Character Codex	Atlas Canvas	A character profile lists origin, residence, or active places	Ties the person back into the world spatially

6.2 Typed links instead of hardcoded assumptions

A place should not just contain a generic list of characters. The stronger pattern is typed links. For example, a character may be linked to a place as resident, ruler, origin, active in, tied to event in, or other relation types. This is cleaner than storing a vague grab-bag list because it preserves meaning.

6.3 Example user movement through the app

- The user opens a world and lands on its overview.
- They enter Atlas Canvas and click a region.
- The region panel shows a linked capital city and a linked archive document about a civil conflict.
- From the city panel, they jump to a ruler inside Character Codex.
- From the character page, they open the ruler's in-character intro, then follow a quick link to a diary entry in Lore Archive.
- The app still feels like one world because the context remains connected the whole way through.

6.4 Source of truth principle

Structured records should remain the source of truth for structured entities. Lore documents should remain writing. This is not just a technical preference; it is a product clarity rule. When the user thinks of a character, the app should know that character as a character record. When the user thinks of a place, the app should know that place as a place record.

7. Finished-picture user experience

This section describes what the app should feel like from the user's point of view when the concept is fully coherent. It is still concept-focused, but it is explicit enough that another AI or human designer can picture the intended flow.

7.1 Home screen

The home screen is the world index. It should show the user's wormholes as world cards or entries. Each world can display a title, optional cover image, short summary, and metadata such as last modified date. The home screen is not where deep work happens; it is where the user selects the universe they want to enter.

7.2 Opening a world

Opening a wormhole should feel like entering a universe dashboard. The default landing area should be a world overview, not the lore folder tree. The world overview should quickly answer: what this world is, what its tone is, what its major places are, who its important characters are, and what recent or important documents exist.

7.3 Internal navigation

The refined direction is a **dock-style top tab navigation** by default. It may later be movable to the left, right, or bottom, but top docking is the default. This keeps the interface cleaner and less vertically wasteful than a permanent left sidebar while still making the three-system structure obvious.

- Overview
- Atlas Canvas
- Character Codex
- Lore Archive

7.4 Working inside Atlas Canvas

The user sees the map canvas, pans and zooms around, selects a semantic place, and inspects its details in a panel. That panel should not be a dead-end popup. It should expose linked characters and linked lore so that the user can continue deeper into the world.

7.5 Working inside Character Codex

Characters should be browsable through a card grid or list with search and filtering. Opening a character should show the three-layer profile concept: fixed core, optional modules, and freeform expressive content. The page should feel like a coherent codex entry, not a raw form dump.

7.6 Working inside Lore Archive

The user navigates a folder/document structure, opens a lore file, reads or edits it, and can move to related characters or places from that document. The archive should support sustained writing while still feeling connected to the world model.

7.7 Overall feel

- coherent rather than fragmented
- expressive but not chaotic

- structured without feeling sterile
- world-first rather than file-first
- local and personal rather than account-and-platform oriented

8. High-level product architecture

Even though this document is not a build spec, it is still useful to describe the intended conceptual architecture at a high level because that helps both humans and AI systems reason about the app correctly.

Layer	Role in the product	Examples
World layer	Top-level project container	world metadata, cover, overview, system tabs
Structured entity layer	Stores records the app must understand directly	characters, map entities, relation links
Document layer	Stores longform written content	lore notes, diary pages, event docs, chapter notes
Navigation/query layer	Retrieval and cross-system movement	search, related items, quick links, typed connections
Presentation layer	How the app is perceived	dock tabs, card views, panels, map canvas, optional visual accents

The recommended technical picture remains: local-first desktop app, Tauri + React + TypeScript, with structured storage separated from lore content. The important concept is not the exact database syntax. The important concept is that structured truth stays structured and lore writing stays writing.

9. Guardrails, boundaries, and future pressure points

Wormhole becomes weak when every interesting idea is absorbed into the main product. The app needs strong boundaries. This section exists so the concept remains healthy even as future ideas appear.

9.1 Healthy boundaries

- Atlas Canvas does not need to become a full art package.
- Character Codex does not need to become a per-character layout design tool in its earliest meaningful form.
- Lore Archive does not need to recreate every note-taking feature from existing note apps.
- Decorative options should stay secondary to retrieval, clarity, and maintainability.

9.2 Floating ideas that should stay outside the core for now

- built-in character look editor similar to Aseprite
- pixel-art or frame-by-frame animation system
- live2D or advanced avatar rendering support
- heavy AI generation features inside the app itself

These ideas are not automatically bad. They are just separate enough that forcing them into the core product too early would bloat Wormhole and weaken the main value.

9.3 The core philosophy to preserve

Core philosophy

Wormhole should let the user express a world richly, but through enough structure that the world remains navigable, meaningful, and maintainable. Expression without structure becomes chaos. Structure without expression becomes sterile.

10. Condensed reference summary

Topic	Refined answer
What is Wormhole?	A personal universe explorer and story-management app built around worlds rather than generic notes.
How is it organized?	Three primary systems: Atlas Canvas, Character Codex, and Lore Archive.
What gives it identity?	The world is spatially explorable, people are structurally understandable, and deeper writing remains connected but separate.
What keeps it clean?	Clear boundaries between place, person, and writing; typed links; hybrid structure; local-first organization.
What should be resisted?	Turning the app into a general creative everything-suite or shoving every future feature directly into the core.

End of document.